

# GenCoder: A Novel Convolutional Neural Network Based Autoencoder for Genomic Sequence Data Compression

Sheena K. S.  and Madhu S. Nair 

**Abstract**—Revolutionary advances in DNA sequencing technologies fundamentally change the nature of genomics. Today's sequencing technologies have opened into an outburst in genomic data volume. These data can be used in various applications where long-term storage and analysis of genomic sequence data are required. Data-specific compression algorithms can effectively manage a large volume of data. In recent times, deep learning has achieved great success in many compression tools and is gradually being used in genomic sequence compression. Significantly, autoencoder has been applied in dimensionality reduction, compact representations of data, and generative model learning. It can use convolutional layers to learn essential features from input data, which is better for image and series data. Autoencoder reconstructs the input data with some loss of information. Since accuracy is critical in genomic data, compressed genomic data must be decompressed without any information loss. We introduce a new scheme to address the loss incurred in the decompressed data of the autoencoder. This paper proposes a novel algorithm called GenCoder for reference-free compression of genomic sequences using a convolutional autoencoder and regenerating the genomic sequences from a latent code produced by the autoencoder, and retrieving original data losslessly. Performance evaluation is conducted on various genomes and benchmarked datasets. The experimental results on the tested data demonstrate that the deep learning model used in the proposed compression algorithm generalizes well for genomic sequence data and achieves a compression gain of 27% over the best state-of-the-art method.

**Index Terms**—Genomic sequence, DNA, deep learning, autoencoder, FASTA, lossless compression.

## I. INTRODUCTION

THE genome is the complete set of DNA (Deoxyribonucleic acid) of an organism, the genetic material in all living organisms that carry genetic instruction for the development, functioning, growth, and reproduction of all known organisms. DNA is a polymer, and its monomer is called a nucleotide which is made up of four chemical building blocks called bases, adenine (A), guanine (G), cytosine (C), and thymine (T) [1], [2]. The sequencing process is a laboratory technique that determines the

order of nucleotide bases in entire genomes or specific genomic regions of interest in DNA or RNA. Next-generation sequencing (NGS) is an excessively parallel sequencing technology that offers ultra-high throughput, scalability, and speed. NGS has revolutionized the biological sciences, allowing it to carry out more heterogeneous applications and study biological systems at a prominent level [3], [4].

NGS makes large-scale whole-genome sequencing (WGS) accessible for all kinds of researchers. It allows scientists to analyze the entire human genome in a single sequencing experiment. NGS techniques have introduced significant changes in genomic sequencing and sequence data analysis. Different file formats are available for storing Genomic sequence data as there is no consensus on how best to store data; in this work, genomes in FASTA format are examined. FASTA is considered as a near-universal standard in the field of bioinformatics. FASTA is a text-based format in which either nucleotide sequences (DNA or RNA) or amino acid sequences (Protein) are represented using single-character codes [5].

Genomic information and technologies influence many parts of our lives. Genomics now provides a powerful tool in various areas, including medical decisions, food safety, ancestry, and more. Scientists can use genomic sequence data to determine multiple gene functions and regulatory instructions in the DNA. The genomic sequence can be screened for characteristic features of genes. Homologous genomic sequences from different organisms can be compared for evolutionary analysis between species or populations diversity and human evolution. Remarkably, genomic sequencing can reveal changes in a gene that may cause disease. Studies of genomic sequences are helpful in healthcare for diagnosing different diseases like cancer, characterizing antibodies, guiding patient treatment, and designing customized drugs. Genomic sequencing revolutionizes food safety and agriculture, improving agriculture through worthwhile plant and animal breeding and minimizing the risks from widespread disease. Moreover, genomic sequencing can protect and enhance the natural environment for both humans and wildlife [6], [7], [8].

The primary genomic sequence databases are part of the International Nucleotide Sequence Database Collaboration, which incorporates the GenBank at NCBI, the DNA Data Bank of Japan (DDBJ), and the European Nucleotide Archive (ENA). These organizations collaborate to exchange data. NCBI builds GenBank predominantly from submissions of sequence data from authors,

Manuscript received 21 October 2022; revised 19 April 2023; accepted 9 February 2024. Date of publication 15 February 2024; date of current version 5 June 2024. The work of Sheena K. S. was supported by University-JRF fellowship under Grant CS27/2019-20/407 (dated 27/11/2019). (Corresponding author: Sheena K. S.)

The authors are with the Artificial Intelligence & Computer Vision Lab, Department of Computer Science, Cochin University of Science and Technology, Kochi 682022, India (e-mail: sheenaks@cusat.ac.in).

Digital Object Identifier 10.1109/TCBB.2024.3366240

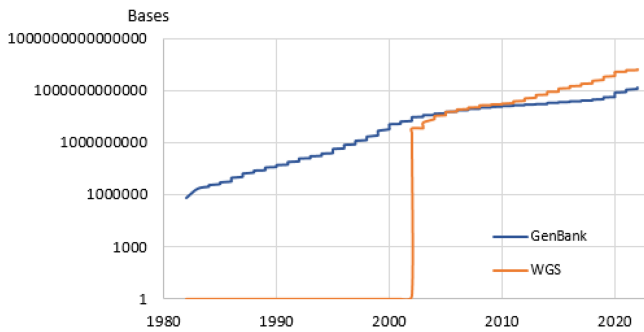


Fig. 1. Growth in number of bases in each release of GenBank.

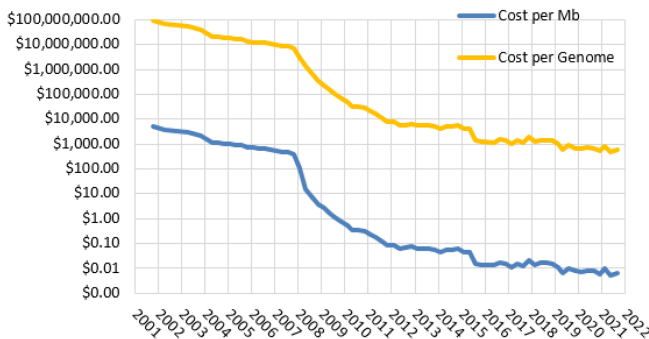


Fig. 2. Cost per raw megabase of DNA sequence and cost per genome. The cost-accounting data are summarized relative to the cost per megabase of DNA Sequence (the cost of determining one million bases) and the cost per genome (the sequencing cost of a human-sized genome).

bulk submissions of whole-genome shotguns (WGS), and other high-throughput data from sequencing centers [9]. A GenBank release occurs every two months. Fig. 1 shows the GenBank growth statistics for both the traditional GenBank divisions and the WGS division, which are available from each release. This data is limited to WGS, Transcriptome Shotgun Assembly (TSA), and Targeted Loci Study (TLS) GenBank records. From 1982 to the present (Data collected on August 2022), the number of bases in GenBank has doubled roughly every 18 months. (<https://www.ncbi.nlm.nih.gov/genbank/statistics>).

The National Human Genome Research Institute (NHGRI) has regularly updated the costs associated with the sequencing centers for DNA sequencing funded by the Institute. The DNA Sequencing cost statistics on their website show the nature of the reductions in the cost of DNA sequencing. The graph in Fig. 2 shows the data since 2001. The NHGRI Genome Sequencing Program (GSP) data is available at ([www.genome.gov/sequencingcostsdata](http://www.genome.gov/sequencingcostsdata)), which shows remarkable improvements in DNA sequencing (Data collected on August 2022). The DNA sequencing costs data graph, Fig. 2, shows a long-term downtrend.

The reductions in the DNA sequencing costs associated with sequencing performed at various centers and improved throughput of sequencing machines result in a plethora of genomic sequencing data. Thus shortly, every human can get their genome sequenced, which can be an inevitable tool in personal health care. The two graphs, Figs. 1 and 2, indicate an evident future explosion of genomic data growth. These exponentially growing

data raise challenges in storing, transmitting, and querying. Storage can be reduced by compression but using a general-purpose compression algorithm is not optimal for genomic data because of the unique characteristics of DNA sequences. The storage and transfer of the enormous amount of genomic data have become a predicament, thus motivating the development of efficient compression tools designed specifically for genomic sequence data. A compression scheme can be lossy or lossless [10]. Lossless compression reconstructs the data from the compressed data without any loss of information by exploiting the statistical redundancy present in the data. Lossy compression allows information loss, and the reconstructed data only approximates the original data, though usually with a significantly improved compression ratio. Lossless data compression is used in applications where deviations from the original data would be unfavourable.

S. Grumbach, F. Tahi opened the research on a data-specific compression algorithm for genomic sequences by introducing Biocompress, a lossless DNA compression algorithm [11] based on regularities, such as the presence of palindromes, in the DNA. After that, numerous compression methods have been coming to the scene, which can be roughly classified into reference-based and reference-free. Reference-based compressors compress multiple genomes by taking one of them as a reference. It exploits the similarity between genomes. The second type of compressor compresses each genomic data file independently.

The algorithm DELIMINATE [12] uses a combination of delta encoding and binary encoding. It separately handles the header and the sequence data. The method compresses sequence data in two phases. Various files generated in the compression process are further compressed using the 7-Zip archiver [13] to generate the final compressed file. It supports FASTA/MultiFASTA file format. It can handle all possible characters in a FASTA file.

MFCompress [14] algorithm compresses genomic sequence based on a probabilistic model that complies with the Markov property. A single finite-context model is used for encoding the header text. Multiple competing finite-context models and arithmetic coding are used to encode the DNA sequences with a forward adaptive coding scheme. XM [15] is another compressor for genomic sequences using a context-based expert predictive model and arithmetic coding. It uses several 'experts' to predict the probability of the next symbol. XM gives a better compression ratio but takes a long computational time. NAF [16] is another reference-free compression tool available for sequence files. In NAF, compression operates by splitting the input into header and sequences. Sequences are concatenated together and then converted into 4-bit coding, which stores two nucleotides per byte. All the resulting data streams are then compressed with the general-purpose compressor zstd [17].

Some compression schemes use neural networks for compressing genomic sequences; the compressor GeCo3 [18] uses a feed-forward artificial neural network trained with stochastic gradient descent. It uses a combination of multiple context models and substitution-tolerant context models of several order depths. DeepZip [19] combines recurrent neural network predictors with an arithmetic coder and losslessly compresses a variety of synthetic data, text and genomic datasets. A predictor

block is used to estimate the conditional probability distribution of a sequence element based on the previously observed symbols. The probability predictor block is modeled as a recurrent neural network (RNN). Arithmetic coding is used for further compression of predicted sequences. Another compression algorithm in [20] applies a deep learning model using a simple deep neural network (DNN) autoencoder for compressing genomes. This method describes a reference-free compression only for the sequence part (not including the header) with character codes 'n', 'a', 'c', 'g' and 't'. The genomic sequence is encoded into a binary sequence, the binary data is then converted to an array, and finally, the array is compressed using the autoencoder. This work contributes an autoencoder based lossy algorithm for genomic sequence.

As genomic data applications span vivid areas, the increasing sequencing rate and lack of standardization in file formats still demand a better sophisticated compression scheme for genomic data. This paper describes a reference-free genomic data compression algorithm that can be used to compress genomic sequences in FASTA formatted files. Our algorithm relies on a deep learning model with an autoencoder using one-dimensional convolutional layers for genomic sequence compression. The deep neural network is made up of multiple layers of interconnected nodes, where each node performs a specific computation on the input data. The network can extract increasingly complex features and patterns from the data without manual feature engineering, making it particularly useful for analysing large datasets. Cheap learning pertains to the idea of using simpler neural networks with solely a handful of layers, more lightweight models that can be trained quickly [21]. By combining the strengths of both deep and cheap learning, it is possible to develop more efficient and cost-effective machine learning solutions for a broader range of applications and problems.

We summarize our significant contributions as follows:

- To the best of our knowledge, it is the first time that an autoencoder with convolutional layers is used for lossless genomic data compression.
- A better compression ratio is achieved compared to other state-of-the-art methods.
- The compression time is shorter and offers faster decompression.
- The proposed method can handle extended alphabets and considers non-ATGC character codes such as N, R, S, W, Y, M, B and K.

In Section II, the proposed method is introduced, including the pre-processing of genomic sequences, the proposed CNN autoencoder, genomic sequence encoding, header compression and decompression. Section III outlines datasets, experimental set-up and results. Lastly, the discussion and conclusion are presented in Section IV.

## II. METHODS

This paper presents GenCoder, a novel lossless compression method for genomic sequences using an autoencoder to learn the statistical redundancies in the sequence data [22]. An autoencoder with convolutional layers has been used for

lossless genomic data compression. GenCoder accepts a FASTA file and compresses it using header and sequence compressors. The header compressor uses the Huffman encoding algorithm for lossless data compression. The objective of the sequence compressor is to transform the input data  $l$  into a lower dimension latent code,  $h$  and then reconstruct the data  $\hat{l}$  from  $h$ . We are taking the residual  $(l - \hat{l})$  information that helps to retrieve the data losslessly and is explained in Section II-G. The reconstruction error between  $l$  and  $\hat{l}$  is measured to evaluate the autoencoder using MSE loss. Finally, the proposed algorithm is evaluated using compression ratio and compression time measures.

The pipeline of the compression process is shown in Fig. 3. Like many algorithms that have already been developed [12], [15], [16], GenCoder compresses sequence and header separately. The algorithm starts with reading a FASTA file and then dividing that into two separate parts, one containing the header part and the other being the sequence part. The sequence part is subjected to pre-processing. The sequence is truncated to fixed-sized subsequences. Then, each subsequence is converted to an array of character codes. The character codes are then transcribed to integer values of  $\{1, 2, 3, 4\}$ . This set of subsequences is used to train the autoencoder. The autoencoder comprises five layers, including two convolutional layers and one fully connected layer. The autoencoder is trained for a minimum loss, and this trained model is used to compress and decompress the sequence part. Some information regarding the input sequence, such as the sequence length, the number of characters in each line, and positions of the non-ATGC character codes, if any, constitutes the side information. The output components of the sequence compressor are further encoded using appropriate entropy techniques. The header part is separately encoded and can be done in parallel to sequence encoding.

A FASTA file begins with a header, and the following lines contain the sequence data. The length of the genomic sequences varies between files. A genomic sequence ideally has character codes A, C, G, and T; however, in FASTA format, sometimes other codes may be present. The sequences are expected to be represented in the standard IUB/IUPAC nucleic acid codes [23]. Thus in this work, we assume a genomic sequence,  $S_o = s_0 s_1 s_2 \dots s_{n_o-1}$  where  $s_i \in \{A, C, G, T, N, R, S, W, Y, M, B, K\}$  and  $n_o = |S_o|$  is the length of the sequence.

### A. Pre-Processing

Pre-processing is performed on the sequence part, which includes:

- 1) ignoring all line-breaks in the sequence and converting all character codes to uppercase.
- 2) Treat all characters not in the alphabet as 'N' (if present).
- 3) Identify patterns of 'N', record the information, and eliminate them.
- 4) locating other non-ATGC characters and subsequently removing them after recording their positions.

GenCoder records the positions of all non-ATGC characters, if present. In some files, long runs of the character 'N' will

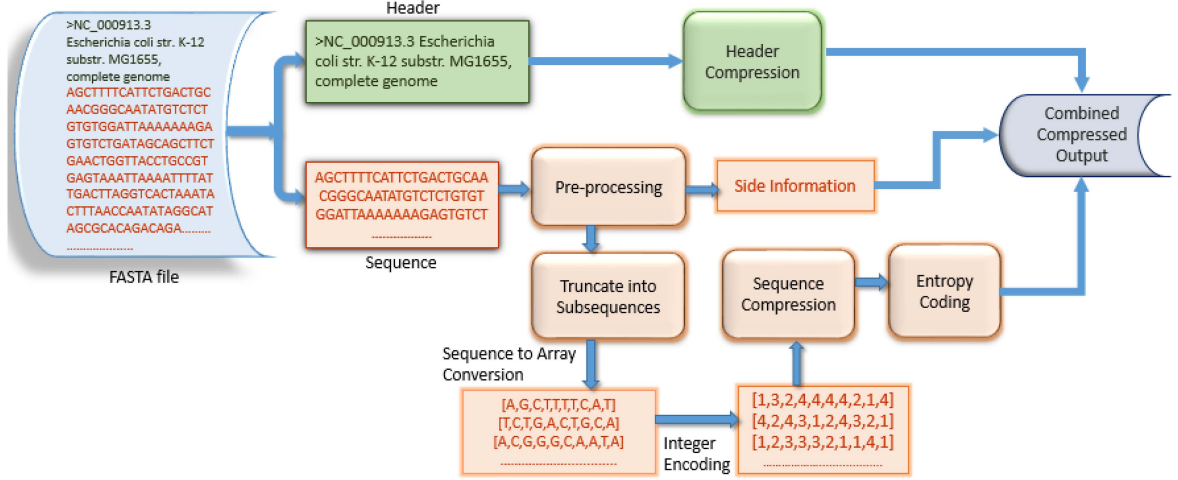


Fig. 3. Pipeline of the proposed compression process.

be present; then, starting location and length of each run are recorded. The method then removes all non-ATGC characters from the input sequence. GenCoder keeps separate lists for storing information about each non-ATGC character that forms the side information. The side information also records the length  $n_o$  and the number of character codes in a single line in the input file. The resulting sequence after pre-processing contains only character codes A, C, G, and T.

Let a genomic sequence after pre-processing be  $S = s_0 s_1 s_2 \dots s_{n-1}$ , where  $s_i \in \{A, C, G, T\}$  and  $n = |S|$  is the length of the sequence. The genomic sequence  $S$  is truncated into a set of subsequences  $l$  of fixed length  $L$ , such that  $|l^i| = L$  where  $0 \leq i \leq k$

$$l = l^0, l^1, l^2, \dots, l^k \quad (1)$$

Then,  $l^0 = s_0 s_1 s_2 \dots s_{L-1}$ ,  $l^1 = s_L s_{L+1} s_{L+2} \dots s_{2L-1}$ ,  $\dots$ ; where  $k = \text{floor}(n/L)$ . Here,  $k+1$  indicates the number of subsequences. The last sequence  $l^k$  is padded to get the length equal to  $L$  so that it becomes  $l^k = s_{kL} s_{kL+1} s_{kL+2} \dots s_{(k+1)L-1}$ .

### B. Integer Encoding

After the pre-processing, the set of subsequences  $l$  is converted into a two-dimensional matrix of dimension  $(k+1) \times L$ , where each row represents a subsequence. Machine learning models require numerical values for feature matrices. Thus, all the character codes are encoded to integer values. Each character code A, C, G, and T is encoded in this approach as 1, 2, 3, and 4, respectively. After normalization A, C, G, and T become 0.25, 0.5, 0.75, and 1.0, respectively.

### C. The Convolutional Neural Network Autoencoder

For the compression and decompression process of the sequence part, the role of the autoencoder is crucial.

Autoencoder is an artificial neural network that can be used to encode the input data into a latent representation of meaningful features and reconstruct the input data back from the latent

representation. The reconstructed data will be a degraded version of the input data. The encoding and decoding processes of the autoencoder with a single hidden layer are mathematically modeled as follows.

Given the input data  $l \in \mathbb{R}^n$ , the encoded data  $h \in \mathbb{R}^m$  for  $m \ll n$

$$h = \sigma(W_{Encoder}l + b_{Encoder}) \quad (2)$$

where  $\sigma: \mathbb{R}^m \rightarrow \mathbb{R}^m$  is a nonlinear activation function (e.g., rectified linear unit (ReLU), sigmoid or hyperbolic tangent), and  $W_{Encoder} \in \mathbb{R}^{m \times n}$  and  $b_{Encoder} \in \mathbb{R}^n$  are weights and bias of hidden layer, respectively. Then, the output of the autoencoder  $\hat{l} \in \mathbb{R}^m$  is given by

$$\hat{l} = \sigma(W_{Decoder}h + b_{Decoder}) \quad (3)$$

A loss function can be used to estimate the loss of the model (e.g., mean absolute error Loss and mean squared error). The reconstruction loss function measures the distance between the output of the *Decoder* and the input of the *Encoder*; then, the learning algorithm is used to adapt all the weights and biases such that  $loss \rightarrow 0$ . A multilayer autoencoder has more than one hidden layers connecting the input layer to the output layer. In forward propagation, the input is transferred and processed for feature extraction. Simultaneously, model parameters, such as weights and biases, have been updated during backpropagation. The forward and backward propagations were applied in the training phase to obtain the model parameters' values to reduce the loss function [24].

A convolutional layer produces a feature map by taking the convolution operation between the input (or the previous feature map) vector and the feature extractor filter.

**Definition 1 (Discrete Time 1-D Convolution):** [25] Let  $f$  and  $g$  be two discrete functions. The convolution  $f * g$  of  $f$  and  $g$  is defined as

$$(f * g)(x) = \sum_{u=-\infty}^{\infty} f(u)g(x-u) \quad (4)$$

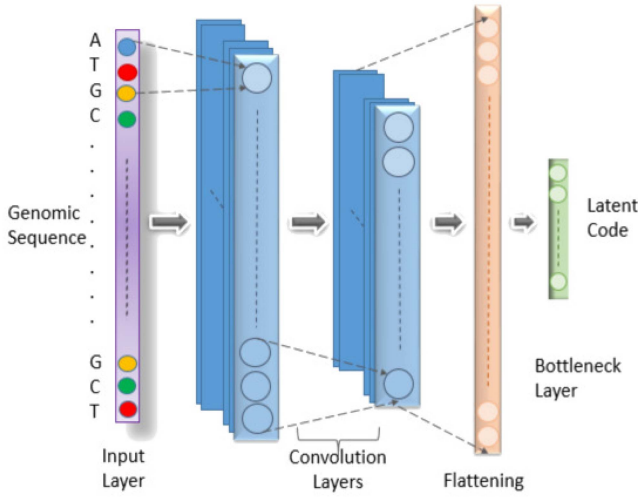


Fig. 4. Encoder unit architecture of the proposed autoencoder.

The convolution operation of two functions represents the amount of overlap between the two functions.

#### D. The Proposed Convolutional Neural Network Autoencoder

The CNN Autoencoder consists of two parts:

##### 1) Encoder block ( $S_\phi : l^i \rightarrow h^i$ )

Input of encoder is  $l^i = s_0 s_1 s_2 \dots s_{L-1}$ . The model learns a latent representation of the input data and reduces the dimensions, and compresses the data into a latent code  $h^i$ , a low dimensional representation of the input.

##### 2) Decoder block ( $S_\psi : h^i \rightarrow \hat{l}^i$ )

Decoder maps  $h^i$  to the reconstruction  $\hat{l}^i$  of the same shape as  $l^i$ . Reconstructed subsequence,  $\hat{l}^i = S_\psi(S_\phi(l^i))$   
 $S_\phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$   $S_\psi : \mathbb{R}^m \rightarrow \mathbb{R}^n$  where  $m \ll n, l^i \in \mathbb{R}^n, h^i \in \mathbb{R}^m, \hat{l}^i \in \mathbb{R}^n$ .

The loss function is calculated for each element in the reconstructed sequence  $\hat{l}_j^i$  of the subsequence  $l^i$ .

$$Loss(S_\psi, S_\phi) = \frac{1}{L} \sum_{j=1}^L (l_j^i - S_\psi(S_\phi(l_j^i)))^2 \quad (5)$$

Back propagation is used to optimize the network parameters iteratively based on the gradient of the loss function with respect to weight parameters. An optimization algorithm is needed to achieve the minimum of the loss function in (5). There are numerous types of gradient-based optimization methods, and in this work Nadam optimizer [26] is used.

Fig. 4 provides the architecture of the encoder unit of constructed CNN-based deep learning model. The decoder unit is the reverse of the encoder unit. The input data has been pre-processed in such a way that each record contains fixed length  $L$  character codes. The proposed architecture is a feed-forward network, and its encoder unit has two CNN layers followed by one dense or fully connected layer.

- CNN layers: A one-dimensional convolutional layer with a filter size of 32 as feature detectors form the first layer. A kernel size of 3 is applied to each feature detector. This first

layer helps the model to learn basic features. The second layer is also a convolutional layer that allows the model to learn more complex features with a filter size of 64 and kernel size of 3. In this feed forward network, the first CNN layer's output is fed to the input of the second layer. Let  $x_k^r$  be the input of the  $k^{th}$  neuron on the layer  $r$ . The value  $x_k^r$  can be expressed as

$$x_k^r = \sigma \left( \sum_{i=1}^{r-1} 1Dconv(w_{ik}^{r-1}, y_i^{r-1}) + b_k^r \right) \quad (6)$$

where  $b_k^r$  is the bias of the  $k^{th}$  neuron on layer  $l$  and  $y_i^{r-1}$  is the output of  $i^{th}$  neuron on layer  $r-1$ .  $w_{ik}^{r-1}$  is the 1-D kernel from the  $i^{th}$  neuron on layer  $r-1$  and  $\sigma$  represent the activation function.

- Flatten layer: A flattening layer is needed to convert all the resultant feature maps after convolution into a single long linear vector to feed the fully connected networks of the bottleneck layer.
- Bottle neck layer: The last layer is a fully connected layer that compiles the data extracted by previous convolutional layers to form the final output. One fullyconnected layer with reduced number of neurons acts as the bottleneck. The bottleneck layer with  $P$  neurons gives a latent code size of  $P$ . It extracts the most relevant features from the input according to the compressibility of the neural network. The choice of latent code size is a trade-off between compressibility and reproducibility [27]. The number of layers in the bottleneck is limited to one; otherwise, it requires a higher amount of RAM for larger subsequences. All the parameters in the design of the autoencoder are empirically determined.

#### E. Genomic Sequence Encoding and Decoding

As depicted in Fig. 5, at the Sender side, the compression as well as the reconstruction of sequence values is performed, and the residual is calculated. At the Receiver side, sequence values are reconstructed and residual is added to it.

- 1) Sender side: To decompress the sequence losslessly, we include both encoder and decoder units of autoencoder at the Sender side. The encoder unit encodes each subsequence  $l^i$  to the latent code,  $h^i$  (the compressed representation of  $l^i$ ). Then the decoder unit reconstructs the sequence  $\hat{l}^i$  from  $h^i$ . Since autoencoder-based compression involves loss of information,  $\hat{l}^i$  is not exactly similar to  $l^i$ . However, a loss cannot be acceptable in the case of genomic sequence. Hence, the difference between the original input sequence and the reconstructed output sequence is calculated, forming the residual information  $\mathcal{R}^i$  and is given by  $l^i - \hat{l}^i$ . As the autoencoder can accurately reconstruct the data more than 80% that our result shows, this residual information will be a sparse vector; which can be compressed effectively by suitable compression methods. The residual information and the latent code corresponding to the entire input subsequence set  $l$ , are denoted by  $\mathcal{R}$ , and  $h$ , respectively; and are stored as two-dimensional matrices. The matrix  $\mathcal{R}$  has dimension  $(k+1) \times L$ , where each row

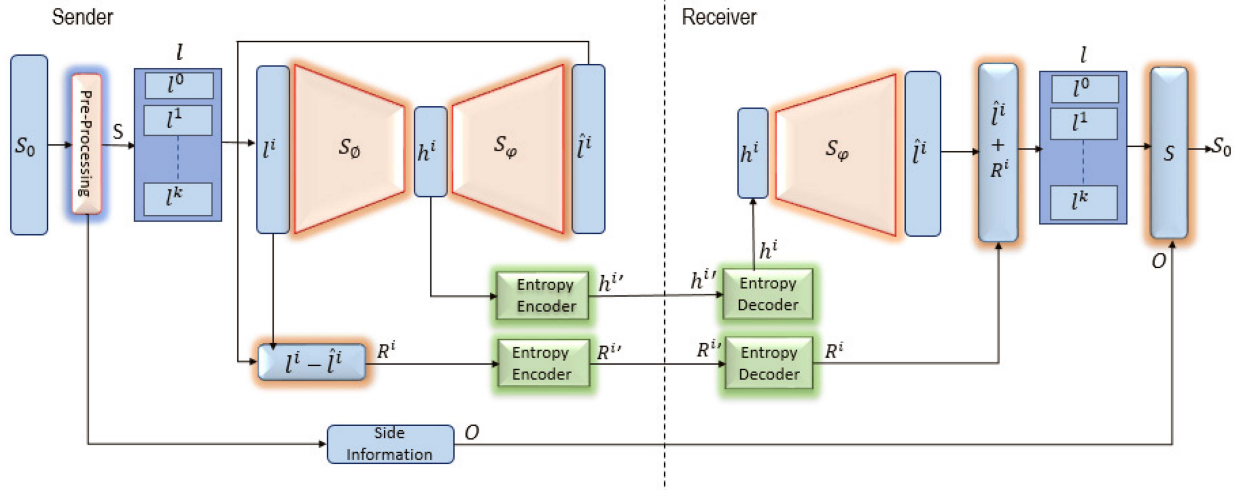


Fig. 5. High-level illustration of the sequence compression and decompression process involved in the Sender and the Receiver side of GenCoder.

represents the residual information of each subsequence. The matrix  $h$  with dimension  $(k+1) \times P$  is the latent code of the entire input subsequences. The  $\mathcal{R}$  and  $h$  are further compressed to  $\mathcal{R}'$  and  $h'$ , respectively, by the entropy coding technique. Each entropy coding involved in this algorithm is described in Section II-E1. Finally, the sequence compressor of GenCoder comprehends the output as the residual information ( $\mathcal{R}'$ ), the latent code ( $h'$ ) and the side information ( $O$ ) as the overhead.

- 2) Receiver side: The receiver part is comprised of the decoder unit of the autoencoder and entropy decoders for the residual information and the latent code. The receiver decompresses each component of the received data using the corresponding decoder. The decoder unit accepts the latent code as the input and reconstructs the entire subsequences. The results are then converted to a two-dimensional integer matrix representation, denoted by  $\hat{l}$ . The residual information  $\mathcal{R}$  is then added to  $\hat{l}$ . The resultant matrix is converted to the corresponding sequence after post-processing. To retrieve the entire data losslessly we require the side information that must be communicated with the receiver side, which includes sequence length ( $n_0$ ), number of character codes in a single line, list of each non-ATGC character codes along with indices of their occurrences in the sequence. These may be considered as an overhead  $O$  in this compression scheme. Using the overhead part, insert all non-ATGC characters of FASTA in their original position, thereby retrieving data losslessly.

1) *Entropy Coding*: In this section, we describe the methods for encoding the residual information matrix generated during sequence coding and the latent code produced by the autoencoder's encoder unit. In a sparse matrix, most elements are zeros. Hence, it makes sense to use only non-zero values to perform operations. By doing so, the memory required for data storage can be minimized.

*Encoding Residual Information Matrix*: As the residual information matrix will be sparse, it can be compressed effectively

using sparse array compression techniques. In GenCoder, Compressed Sparse Row (CSR) algorithm is applied to compress the sparse residual matrix. The compressed sparse row (CSR) or compressed row storage (CRS) or Yale format represents a matrix by three (one-dimensional) arrays that contain non-zero values, as (row index, column index, value) in row order. This format allows fast row access and matrix-vector operations [28].

*Encoding Latent Code*: The latent code produced by the autoencoder's encoder unit is a floating point array. For lossless compression of the floating point array, every bit of each floating-point number must be precisely preserved without any loss in accuracy by using an efficient streaming fpzip compressor, which usually provides a data reduction depending on data precision and smoothness. fpzip is a BSD-licensed open-source library for lossless or lossy compression of large multidimensional floating-point arrays [29], [30].

#### F. Header Compressor

The header part of the FASTA file consists of a single-line description, which includes a unique library accession number and may also contain additional information and starts with the symbol '>'. So this part is compressed using the Huffman algorithm [31], a variable length prefix coding technique. The compressed code is denoted by  $H$ . Thus, the total size of the compressed data of GenCoder is given by

$$\text{Compressed size} = \text{size}(H) + \text{size}(\mathcal{R}') + \text{size}(h') + \text{size}(O) \quad (7)$$

where  $H$  is the code for header,  $\mathcal{R}'$  is the compressed residual information,  $h'$  the compressed latent code and  $O$  is the side information.

#### G. Decompression

The decompression process is performed at the receiver side in reverse that of the compression process, which consists of three steps. First, the header, the latent code and the residue matrix ( $h$

TABLE I  
DNACORPUS

| Seq.ID | Species name                | Type                 |
|--------|-----------------------------|----------------------|
| OrSa   | Oriza sativa                | Eukaryota, plant     |
| HoSa   | Homo sapiens                | Eukaryota, animalia  |
| GaGa   | Gallus gallus               | Eukaryota, animalia  |
| DaRe   | Danio rerio                 | Eukaryota, animalia  |
| DrMe   | Drosophila miranda          | Eukaryota, animalia  |
| EnIn   | Entamoeba invadens          | Eukaryota, amoebozoa |
| ScPo   | Schizosaccharomyces pombe   | Eukaryota, fungi     |
| PlFa   | Plasmodium falciparum       | Eukaryota, protozoan |
| EsCo   | Escherichia coli            | Bacteria             |
| HePy   | Helicobacter pylori         | Bacteria             |
| AeCa   | Aeropyrum camini            | Archaea              |
| HaHi   | Haloarcula hispanica        | Archaea              |
| YeMi   | Yellowstone lake mimivirus  | Virus, mimivirus     |
| BuEb   | Bundibugyo ebolavirus       | Virus                |
| AgPh   | Aggregatibacter phage S1249 | Virus, phage         |

TABLE II  
ADDITIONAL SEQUENCES

| Seq.ID | Species name                     | Type                  |
|--------|----------------------------------|-----------------------|
| OrSa11 | Oriza sativa, chr.11             | Eukaryota, plant      |
| HoSa3  | Homo sapiens, chr.3              | Eukaryota, animalia   |
| DaRe2  | Danio rerio, chr.2               | Eukaryota, animalia   |
| SaCe   | Saccharomyces cerevisiae         | Eukaryota, fungi      |
| SaEn   | Salmonella enterica              | Bacteria              |
| AcMi   | Acanthamoeba polyphaga mimivirus | Virus, mimivirus      |
| ObLa   | Oberland virus                   | Virus, partial genome |
| AcPh   | Acinetobacter phage AM24         | Virus, phage          |

and  $\mathcal{R}$ ) are decompressed by corresponding decoders. The side information collected during the pre-processing phase has also been communicated to the receiver. In the second step, the decoder unit of the autoencoder accepts latent code and regenerates the sequence. The output values are subjected to post-processing (inverse of normalization) and are then converted to an integer matrix ( $\hat{l}$ ). The matrix  $\hat{l}$  with dimension  $(k+1) \times L$ , represents the reconstructed matrix of integer values; it is then directly added to  $\mathcal{R}$ . Thus the integer matrix  $l$  of dimension  $(k+1) \times L$  is recovered; note that the matrix  $\mathcal{R} = (l - \hat{l})$ .

Third, the matrix  $l$  is transcoded to DNA character codes 'A', 'C', 'G', and 'T'. Then the matrix is transformed to the DNA sequence; thereby, the initial sequence  $S$  is retrieved. Each non-ATGC character is inserted in the correct positions using the side information. Finally, the genomic sequence is decoded without any loss of information. The sequence length and the number of characters in a single line help to reassemble the sequence. The sequence and the header are assembled into FASTA formatted output file. The resulting file contains the header and the genomic sequence,  $S_o = s_0 s_1 s_2 \dots s_{n-1}$  where  $s_i \in \{A, C, G, T, N, R, S, W, Y, M, B, K\}$  with sequence length  $n_o = |S_o|$  as that of the original input file.

### III. RESULTS

GenCoder can be used as a lossless reference-free genomic sequence compressor for the FASTA file format. We attempt to compare the performance of GenCoder with three state-of-the-art reference-free compressors, NAF, XM, and GeCo3 and one general-purpose compressor, Gzip. All these tools support FASTA sequences. Several quantitative metrics are commonly

used to express the compression algorithm's performance. Different authors reported different performance measures. To have a uniform approach the performance is measured using compressed size (CS) in bytes, compression time (CT) and decompression time (DT) in seconds.

$$\text{Compression ratio (CR)} = \frac{\text{Size of the output stream}}{\text{Size of the input stream}} \quad (8)$$

Compression in bits per base is computed as

$$\text{CB} = \frac{\text{Compressed size in bits}}{\text{Total No of bases in the sequence}} \quad (9)$$

A Compression ratio of 0.7 means that the compressed output data occupies 70% of its primordial size. The compression ratio greater than 1 means an output stream bigger than the input stream, indicating expansion. It can also be called bpb (bit per bit) since it equals the number of bits in the compressed stream needed, on average, to compress one bit in the input stream. In DNA compression methods, it makes sense to talk about bpb (bits per base), the number of bits it takes hold of, on an average, to compress one base in the input stream as defined in the (9), [10], [32]. Comparison between different genomic sequence compressors is challenging because of the file format supported and the alphabet considered. Since different genomic sequences have a different repetitive index and different sequence characteristics, various algorithms perform biased on different datasets.

#### A. Datasets and Experimental Set-Up

A benchmark dataset proposed in [33] as DNACorpus, which includes the genomic sequences of main domains and kingdoms, is used to compare GenCoder with state-of-the-art compression tools. Apart from this, we applied our algorithm to two different human genome assemblies (KOREF\_S1v2.1, GRCh38.p14) in order to understand our approach better. These two genomes have been widely used in most other genomic compressors. The DNACorpus includes the sequences as shown in Table I. In Table I the Seq.ID is used to represent a sequence file. The type defines the kingdom of the species where the sequence has been extracted. The size is in the number of bases in the sequence. In all files, the number of character codes in the alphabet used is 4. The dataset includes chromosome 1 of Oriza sativa, chromosome 4 of reference human genome, chromosome 2 of Gallus Gallus, chromosome 3 of Danio rerio, chromosome 2 of Drosophila miranda and genomes of other species, each file consists only of sequence part.

KOREF is the Korean reference genome, which was constructed with various sequencing technologies. The KOREF\_S1v2.1 is the first chromosome-scale haploid assembly of the Korean reference genome [34]. The previous KOREF assembly has been updated to a new chromosome-level haploid assembly, KOREF\_S1v2.1. The human reference genome, GRCh38, has been updated and recently was patched in 2022 by the Genome Reference Consortium (GRC) as GRCh38.p14 (Genome Reference Consortium Human Build 38 patch release 14). The two genomes are the latest versions downloaded from the website [35].

TABLE III  
COMPRESSED SIZE (CS) AND COMPRESSION RATIO(CR) OBTAINED FOR NAF, GeCo3, XM, GZIP, AND GENCODER FOR DNACORPUS DATASET

| Seq.ID | DNA Sequences<br>Size (B) | NAF         |       | GeCo3       |       | XM          |       | Gzip        |       | GenCoder   |       |
|--------|---------------------------|-------------|-------|-------------|-------|-------------|-------|-------------|-------|------------|-------|
|        |                           | CS (B)      | CR    | CS (B)      | CR    | CS (B)      | CR    | CS (B)      | CR    | CS (B)     | CR    |
| OrSa   | 43,262,523                | 9,992,929   | 0.231 | 8,745,124   | 0.202 | 8,470,212   | 0.196 | 12,247,694  | 0.283 | 6,530,699  | 0.151 |
| HoSa   | 189,752,667               | 43,757,076  | 0.231 | 39,384,515  | 0.208 | 38,940,458  | 0.205 | 53,129,246  | 0.280 | 28,271,842 | 0.149 |
| DaRe   | 62,565,020                | 13,453,230  | 0.215 | 11,502,879  | 0.184 | 11,302,620  | 0.181 | 17,323,427  | 0.277 | 9,354,727  | 0.150 |
| ScPo   | 10,652,155                | 2,715,812   | 0.255 | 2,631,926   | 0.247 | 2,524,147   | 0.237 | 3,095,627   | 0.291 | 1,600,929  | 0.150 |
| EsCo   | 4,641,652                 | 1,205,862   | 0.260 | 1,142,948   | 0.246 | 1,110,092   | 0.239 | 1,341,960   | 0.289 | 716,400    | 0.154 |
| YeMi   | 73,689                    | 17,766      | 0.241 | 17,193      | 0.233 | 16,861      | 0.229 | 20,926      | 0.284 | 13,194     | 0.179 |
| BuEb   | 18,940                    | 4925        | 0.260 | 4803        | 0.254 | 4642        | 0.245 | 5762        | 0.304 | 4698       | 0.248 |
| AgPh   | 43,970                    | 11,284      | 0.257 | 10,977      | 0.250 | 10,711      | 0.244 | 12,961      | 0.295 | 8645       | 0.197 |
| -----  | -----                     | -----       | ----- | -----       | ----- | -----       | ----- | -----       | ----- | -----      | ----- |
| GaGa   | 148,532,294               | 37,297,848  | 0.251 | 34,875,638  | 0.235 | 33,879,211  | 0.228 | 42,244,285  | 0.284 | 22,153,229 | 0.149 |
| DrMe   | 32,181,429                | 8,231,322   | 0.256 | 7,717,519   | 0.240 | 7,538,662   | 0.234 | 9,214,348   | 0.286 | 4,810,045  | 0.149 |
| EnIn   | 26,403,087                | 6,155,141   | 0.233 | 5,263,852   | 0.199 | 5,150,309   | 0.195 | 7,616,039   | 0.288 | 39,32,509  | 0.149 |
| PlFa   | 8,986,712                 | 2,118,124   | 0.236 | 1,992,294   | 0.222 | 1,925,841   | 0.214 | 2,494,880   | 0.278 | 13,39,398  | 0.149 |
| HePy   | 1,667,825                 | 414,259     | 0.248 | 380,539     | 0.228 | 384,071     | 0.230 | 472,589     | 0.283 | 258,494    | 0.155 |
| AeCa   | 159,1049                  | 406,344     | 0.255 | 386,017     | 0.243 | 387,030     | 0.243 | 459,578     | 0.289 | 246,877    | 0.155 |
| HaHi   | 3,890,005                 | 971,459     | 0.250 | 910,858     | 0.234 | 913,346     | 0.235 | 1,115,264   | 0.287 | 600,952    | 0.154 |
| OrSa11 | 29,433,615                | 7,111,680   | 0.242 | 6,054,226   | 0.206 | 5,316,701   | 0.181 | 8799,458    | 0.299 | 4,930,699  | 0.168 |
| HoSa3  | 200,764,129               | 46,898,583  | 0.234 | 419,82,245  | 0.209 | 41,160,059  | 0.205 | 58,897,956  | 0.293 | 29,987,420 | 0.149 |
| DaRe2  | 60,492,639                | 13,949,076  | 0.231 | 12,039,069  | 0.199 | 10,794,911  | 0.178 | 17,792,834  | 0.294 | 9,854,727  | 0.163 |
| ScCe   | 12,310,316                | 3,092,058   | 0.251 | 2,523,376   | 0.205 | 27,83,433   | 0.226 | 39,62,920   | 0.322 | 19,23,487  | 0.156 |
| SalEn  | 5,013,387                 | 1,238,340   | 0.247 | 1,202,018   | 0.240 | 1,203,715   | 0.240 | 1,518,094   | 0.303 | 971,510    | 0.194 |
| AcMi   | 1,039,262                 | 237,733     | 0.229 | 228,723     | 0.220 | 228,259     | 0.220 | 305,673     | 0.294 | 159,061    | 0.153 |
| ObLa   | 14,803                    | 3802        | 0.257 | 3681        | 0.249 | 3654        | 0.247 | 4796        | 0.324 | 4195       | 0.283 |
| AcPh   | 98,567                    | 23,976      | 0.243 | 23,491      | 0.238 | 23,718      | 0.241 | 30,003      | 0.304 | 16,775     | 0.170 |
| Total  | 532,419,119               | 128,149,745 | 0.241 | 115,583,546 | 0.217 | 111,692,921 | 0.210 | 154,928,717 | 0.291 | 81,189,378 | 0.152 |

The proposed method, GenCoder, has been implemented in Python, and the experiments are run over a system with Intel(R) Xeon(R) CPU @ 2.20 GHz, 56,320 KB cache, 12 GB RAM, and NVIDIA Tesla T4 16 GB GPU.

### B. Experimental Results

We compare GenCoder with other reference-free genomic compression algorithms, namely, XM [15], NAF [16], GeCo3 [18], and Gzip. Table III shows the comparison results for compressing DNACorpus and the additional sequences which are listed in Table II. NAF uses different compression levels, the default is 1. Higher levels provide better compression, but are slower. Maximum level is 22, however levels above 19 are slow and use significant amount of RAM. GeCo3 contains many modes of compression with diverse neural network characteristics.

The autoencoder based deep learning model of GenCoder is trained on the sequence to be compressed for multiple epochs (maximum of 50). Once the training is accomplished, the model weights are set aside, to allow its usage during decompression. In DNACorpus first eight sequences are from the eukaryotic domain, and the remaining sequences belong to the prokaryotic domain. The prokaryotic domain includes two sequences of bacteria, two sequences of archaea and three sequences of viruses. The three viruses are hard to compress, especially the BuEb and AgPh. Separate training is conducted with different input parameter for each domain for better results. From eukaryotic domain OrSa, DaRe, ScPo, and HoSa were taken for training with an input parameter  $L = 12,802$  and a latent code size  $P = \frac{L}{25}$ , obtained a training time of 2078.35 seconds. For prokaryotic domain EsCo, YeMi, BuEb and AgPh are used for training the model with  $L = 1282$  and  $P = \frac{L}{25}$ . We have used some additional sequences as listed in Table II for testing the proposed model since some sequences in DNACorpus are taken for training the model. In Table III, NAF uses a compression level of 22 (highest compression level). XM uses the default configuration. GeCo3 uses mode 16. The size of DNA sequences is the same as that of sequence length since each character code needs one-byte storage. Sequences above the dashed line have

been taken for training and were exempted from calculating the total. According to the results from Table III, XM is the best among the four existing compressors, based on the compression ratio, XM produces a compressed output that occupies 21% (106.52 MB) of its original size (507.75 MB). The experimental results show that GenCoder significantly outperforms XM as its output contains only 15.2% of the input data size. Fig. 6 depicts the compression obtained in bits per base, GenCoder needs only 1.220 bits to represent a single base (character code) in average whereas Gzip needs 2.328 bits. The alphabet size of all the sequences in DNACorpus is four; therefore, the ideal compression is two bpb. As Gzip is a general-purpose compressor, not able to provide efficient compression results for DNA sequences. Comparing GenCoder against the other compressors, the compression gain is 27% (XM), 37% (NAF), 30% (GeCo3) and 47% (Gzip). GenCoder obtained the worst results, 1.984 bpb and 2.267 bpb, for the sequence BuEb and ObLa, respectively; since both are hard-to-compress genome and their sequence length is too short compared to the other sequences.

From Fig. 7 we can observe that NAF has the shortest compression time even if it yields a higher compression ratio than GenCoder. Although XM achieves a better compression performance,  $CB = 1.678$  and  $CR = 0.210$ , it consumes more compression time as shown in Fig. 7. Regarding the time measures of GenCoder, from Fig. 8, it can be observed that the decompression process is faster than the compression process.

Table IV shows the result of compressing GRCh38.p14. The results are listed by chromosomes, with which we can compute the overall compression of the whole genome. Since chromosome 5 was taken for the training, it was excluded from measuring the total. The overall compression results obtained for the genome are  $CB = 1.126$  and  $CR = 0.139$ , the total number of bases in the whole genome except chromosome 5 is 2,913,494,466, and the CS of sequences in bytes is 410,034,140. All the character codes present in the input sequence have been preserved. Each sequence file in the genome has come with header data. So header compression has also been performed and added to the total compressed size.

Table V shows the analysis of compressing KOREF\_S1v2.1. The total gives the overall compression of the whole genome.

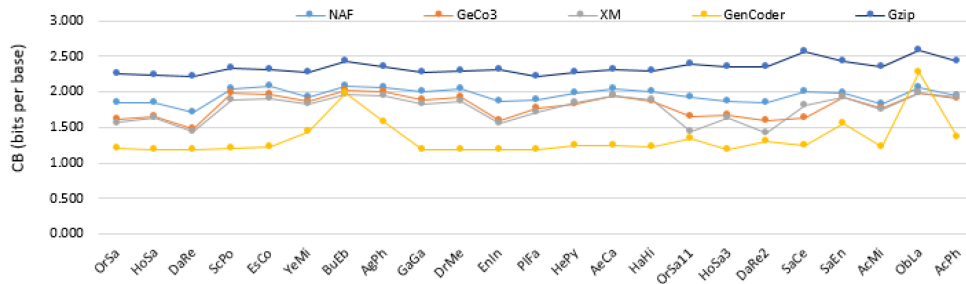


Fig. 6. Compression in bits per base (CB) obtained for NAF, GeCo3, XM, Gzip, and GenCoder.

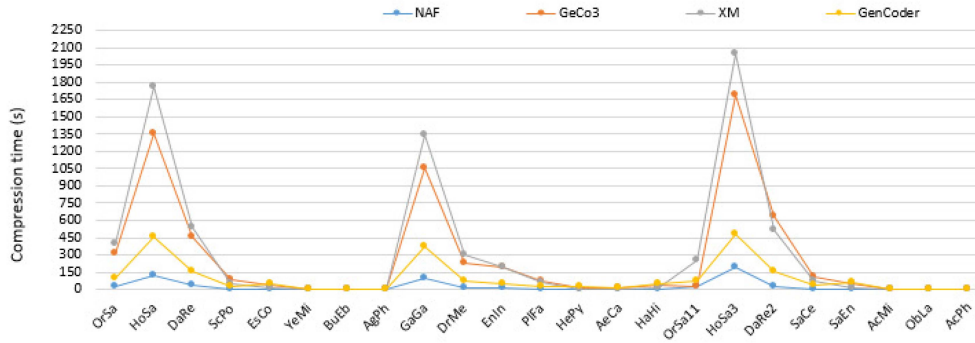


Fig. 7. Compression time (CT) obtained for NAF, GeCo3, XM, and GenCoder.

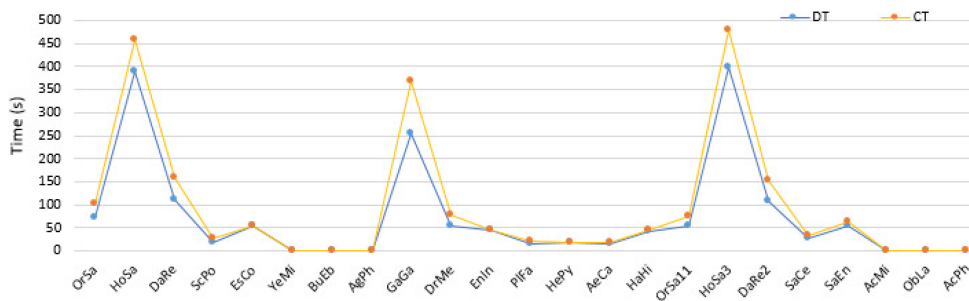


Fig. 8. Decompression time (DT) and Compression time (CT) obtained for GenCoder.

Chromosome 1 was excluded from measuring the total since it has been taken for training. The compression results obtained for the genome are  $CB = 1.192$  and  $CR = 0.147$  while preserving all the character codes in the original input sequences. The total number of bases in the whole genome except chromosome 1 is 2,613,923,758, and the CS of sequences is 389,530,686 bytes. Header compression has also been performed and added to the total compressed size.

The autoencoder was trained using the sequence HoSa of DNACorpus, with input parameters  $L = 18,096$ , batch size = 32, latent code size  $P = \frac{L}{30}$ . The sequence was truncated into subsequences, which were then pre-processed and obtained a set of fixed-length subsequences. From this set of subsequences, 80% of the samples were used for training and 20% for validation. The MSE loss function was employed, and the autoencoders' reconstruction accuracy was calculated as the average of the squared differences between the input and the output data. The

learning curves in Figs. 9 and 10 show the accuracy and loss of the autoencoder, respectively. The learning curves obtained for validation and training data converge at similar values and the smaller gaps indicate that the model generalizes better. The training accuracy can vary depending on the autoencoder model and the input data.

Naturally, the selection of subsequence length  $L$  impacts compression performance. For longer sequences like human chromosomes  $L$  could be a larger value as it seems to be a better choice to get the entire sequences compressed in a short time span. However for short sequences like bacterial genomes, small  $L$  values are preferable. A considerably larger value for  $L$ , even with a latent code size,  $P = \frac{L}{30}$  gave better reconstruction accuracy. Large  $L$  values consume more amount of RAM. Still, for some sequences, an increased padding length may be required to fill the last subsequence, which may affect the compression performance. But a short-length subsequence is not necessarily

TABLE IV  
HOMO SAPIENS GENOME: COMPRESSION RESULTS OF GRCh38.p14

| Chr.No. | Size(MB) | CS(MB) | CT(s) | DT(s) |
|---------|----------|--------|-------|-------|
| 1       | 240.39   | 32.74  | 440   | 357   |
| 2       | 233.86   | 32.74  | 440   | 357   |
| 3       | 191.47   | 28.14  | 369   | 294   |
| 4       | 183.67   | 26.95  | 356   | 284   |
| 5       | 175.29   | 25.82  | 328   | 260   |
| 6       | 164.93   | 24.16  | 320   | 258   |
| 7       | 153.86   | 22.59  | 298   | 239   |
| 8       | 140.15   | 20.57  | 272   | 216   |
| 9       | 133.63   | 17.31  | 225   | 179   |
| 10      | 129.19   | 18.93  | 251   | 198   |
| 11      | 130.44   | 19.11  | 252   | 201   |
| 12      | 128.69   | 18.92  | 247   | 197   |
| 13      | 110.43   | 13.92  | 183   | 145   |
| 14      | 103.36   | 12.87  | 170   | 135   |
| 15      | 98.48    | 12.87  | 156   | 121   |
| 16      | 87.23    | 11.62  | 153   | 121   |
| 17      | 80.39    | 11.79  | 156   | 124   |
| 18      | 77.61    | 11.38  | 150   | 116   |
| 19      | 56.60    | 8.31   | 112   | 88    |
| 20      | 62.23    | 9.09   | 122   | 96    |
| 21      | 45.10    | 5.70   | 78    | 60    |
| 22      | 49.07    | 5.57   | 76    | 59    |
| X       | 150.67   | 22.01  | 289   | 229   |
| Y       | 55.26    | 3.76   | 50    | 33    |
| Total   | 2806.72  | 416.86 | 5163  | 4106  |

TABLE V  
HOMO SAPIENS GENOME: COMPRESSION RESULTS OF KOREF\_S1v2.1

| Chr.No. | Size(MB) | CS(MB) | CT(s) | DT(s) |
|---------|----------|--------|-------|-------|
| 1       | 212.33   | 31.26  | 424   | 348   |
| 2       | 227.56   | 33.47  | 470   | 389   |
| 3       | 188.24   | 27.68  | 386   | 320   |
| 4       | 180.76   | 26.58  | 363   | 299   |
| 5       | 170.48   | 25.07  | 338   | 279   |
| 6       | 162.29   | 23.88  | 321   | 265   |
| 7       | 148.76   | 21.88  | 292   | 241   |
| 8       | 136.19   | 20.03  | 267   | 221   |
| 9       | 108.64   | 15.98  | 217   | 181   |
| 10      | 126.11   | 18.55  | 247   | 205   |
| 11      | 126.28   | 18.58  | 248   | 205   |
| 12      | 125.78   | 18.50  | 249   | 207   |
| 13      | 92.20    | 13.65  | 180   | 150   |
| 14      | 84.80    | 12.48  | 166   | 138   |
| 15      | 77.46    | 11.40  | 152   | 128   |
| 16      | 73.55    | 10.82  | 145   | 122   |
| 17      | 75.60    | 11.13  | 149   | 125   |
| 18      | 72.91    | 10.73  | 143   | 121   |
| 19      | 54.35    | 8.00   | 106   | 91    |
| 20      | 60.03    | 8.83   | 118   | 100   |
| 21      | 31.68    | 4.66   | 63    | 56    |
| 22      | 32.30    | 4.76   | 65    | 57    |
| X       | 144.89   | 21.40  | 284   | 235   |
| Y       | 23.14    | 3.41   | 48    | 44    |
| Total   | 2523.99  | 371.48 | 5017  | 4179  |

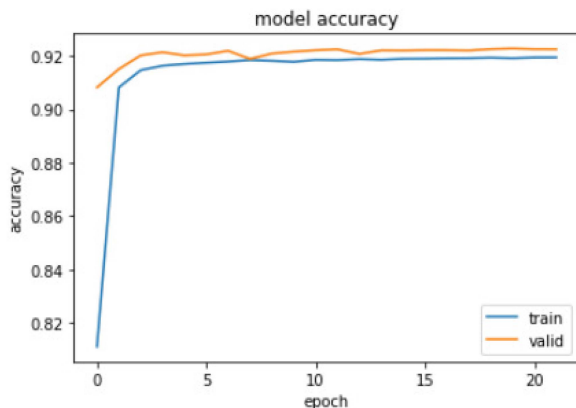


Fig. 9. Training and validation accuracy for the sequence HoSa.

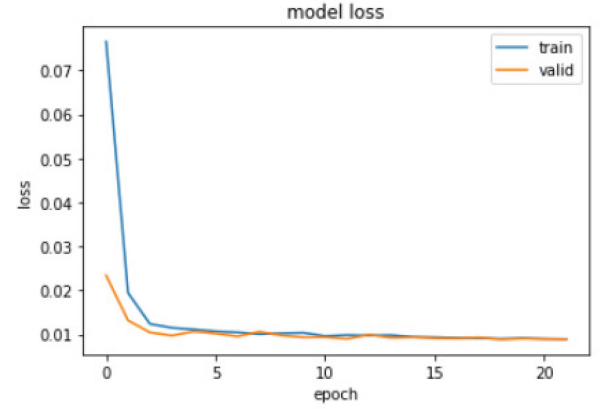


Fig. 10. Training and validation loss for the sequence HoSa.

the best choice. So, one of the future works is to optimize the value of  $L$  for the best compression performance.

#### IV. DISCUSSION AND CONCLUSION

Our goal in this work was to design an algorithm that maximizes the compression performance and retrieves sequences without any information loss within a reduced decompression time. Autoencoders are a standard tool for training neural network algorithms to perform feature extraction and dimensionality reduction. The sequence compression part of our algorithm depends on the autoencoder. Thus the system performance primarily relies on the autoencoder, but it comes with a lot of applicational errors.

Since autoencoder is an unsupervised technique that learns from its own data, it needs considerable training data to generate valuable results. So training the network needs much more time and a significant amount of RAM. If it is trained for genomes of only a specific kingdom, it may not generalize well for genomes of a different kind as it seems to be more data-specific. Moreover, autoencoders are lossy compressors, and compression degradation affects system performance in a significant way if the loss is high.

A typical autoencoder's bottleneck layer consists of multiple layers with a progressively reducing number of neurons and pooling layers for compressing the original input data to latent code. The resulting latent code may be missing essential features of the original data if the bottleneck layer is too constricting. This problem can be circumvented by testing reconstruction accuracy for varying sizes of the bottleneck layer. Different design architecture and hyper-parameters may improve the model further.

We have presented a novel compression algorithm GenCoder which uses an autoencoder-based deep learning model for sequence compression. Even though the autoencoder's reconstruction is lossy, we have proposed a method to overcome that loss without degrading the compression performance. GenCoder can also be adaptable to compress RNA and protein sequences. Our algorithm is shown to perform better than the other DNA compressors reported in the literature.

## REFERENCES

- [1] J. D. Watson and F. H. Crick, "The structure of DNA," in *Cold Spring Harbor Symposia on Quantitative Biology*, vol. 18. Cold Spring Harbor, NY, USA: Cold Spring Harbor Laboratory Press, 1953, pp. 123–131, doi: [10.1101/sqb.1953.018.01.020](https://doi.org/10.1101/sqb.1953.018.01.020).
- [2] R. R. Sinden, *DNA Structure and Function*. Houston, TX, USA: Gulf Professional Publishing, 1994.
- [3] J. Batley and D. Edwards, "Genome sequence data: Management, storage, and visualization," *BioTechniques*, vol. 46, pp. 333–336, 2009, doi: [10.2144/000113134](https://doi.org/10.2144/000113134).
- [4] G. M. Church and W. Gilbert, "Genomic sequencing," in *Proc. Nat. Acad. Sci. USA*, vol. 81, pp. 1991–1995, 1984, doi: [10.1073/pnas.81.7.1991](https://doi.org/10.1073/pnas.81.7.1991).
- [5] H. Zhang, *Overview of Sequence Data Formats*. New York, NY, USA: Springer, 2016, pp. 3–17, doi: [10.1007/978-1-4939-3578-9\\_1](https://doi.org/10.1007/978-1-4939-3578-9_1).
- [6] E. R. Mardis, "DNA sequencing technologies: 2006–2016," *Nature Protocols*, vol. 12, pp. 213–218, 2017, doi: [10.1038/nprot.2016.182](https://doi.org/10.1038/nprot.2016.182).
- [7] H. Buermans and J. den Dunnen, "Next generation sequencing technology: Advances and applications," *Biochimica et Biophysica Acta Mol. Basis Dis.*, vol. 1842, pp. 1932–1941, 2014, doi: [10.1016/j.bbdis.2014.06.015](https://doi.org/10.1016/j.bbdis.2014.06.015).
- [8] D. Gountia and S. Roy, "Design-for-trust techniques for digital microfluidic biochip layout with error control mechanism\*A preliminary version of this paper appeared in the Proc. of IEEE region 10 symposium (TEN-SYMP), 2019 [1]," *IEEE/ACM Trans. Comput. Biol. Bioinf.*, vol. 19, no. 3, pp. 1570–1582, May/Jun. 2022, doi: [10.1109/TCBB.2021.3054622](https://doi.org/10.1109/TCBB.2021.3054622).
- [9] NIH. [Online]. Available: <https://www.ncbi.nlm.nih.gov>
- [10] D. Salomon, *Data Compression: The Complete Reference*. New York, NY, USA: Springer, 2006.
- [11] S. Grumbach and F. Tahi, "Compression of DNA sequences," in *Proc. Data Compression Conf.*, 1993, pp. 340–350, doi: [10.1109/DCC.1993.253115](https://doi.org/10.1109/DCC.1993.253115).
- [12] M. H. Mohammed, A. Dutta, T. Bose, S. Chadaram, and S. S. Mande, "Delimitate a fast and efficient method for loss-less compression of genomic sequences: Sequence analysis," *Bioinformatics*, vol. 28, pp. 2527–2529, 2012, doi: [10.1093/bioinformatics/bts467](https://doi.org/10.1093/bioinformatics/bts467).
- [13] 7-zip file archiver. [Online]. Available: <https://www.7-zip.org>
- [14] A. Pinho and D. Pratas, "MFCompress: A compression tool for FASTA and multi-FASTA data," *Bioinformatics*, vol. 30, pp. 117–118, 2014, doi: [10.1093/bioinformatics/btt594](https://doi.org/10.1093/bioinformatics/btt594).
- [15] M. Cao, T. Dix, L. Allison, and C. Mears, "A simple statistical algorithm for biological sequence compression," in *Proc. Data Compression Conf.*, 2007, pp. 43–52, doi: [10.1109/DCC.2007.7](https://doi.org/10.1109/DCC.2007.7).
- [16] K. Kryukov, M. T. Ueda, S. Nakagawa, and T. Imanishi, "Nucleotide Archival Format (NAF) enables efficient lossless reference-free compression of DNA sequences," *Bioinformatics*, vol. 35, pp. 3826–3828, 2019, doi: [10.1093/bioinformatics/btz144](https://doi.org/10.1093/bioinformatics/btz144).
- [17] Zstandard: ZSTD. [Online]. Available: <https://github.com/facebook/zstd>
- [18] M. Silva, D. Pratas, and A. J. Pinho, "Efficient DNA sequence compression with neural networks," *Gigascience*, vol. 9, 2020, Art. no. gaa119, doi: [10.1093/gigascience/giaa119](https://doi.org/10.1093/gigascience/giaa119).
- [19] M. Goyal, K. Tatwawadi, S. Chandak, and I. Ochoa, "DeepZip: Lossless data compression using recurrent neural networks," in *Proc. Data Compression Conf.*, 2019, pp. 575–575, doi: [10.1109/DCC.2019.00087](https://doi.org/10.1109/DCC.2019.00087).
- [20] Z. N. Absardi and R. Javidan, "A fast reference-free genome compression using deep neural networks," in *Proc. Big Data Knowl. Control Syst. Eng.*, 2019, pp. 1–7, doi: [10.1109/BdKCSSE48644.2019.9010661](https://doi.org/10.1109/BdKCSSE48644.2019.9010661).
- [21] H. W. Lin, M. Tegmark, and D. Rolnick, "Why does deep and cheap learning work so well?," *J. Statist. Phys.*, vol. 168, pp. 1223–1247, 2017, doi: [10.48550/arXiv.1608.08225](https://doi.org/10.48550/arXiv.1608.08225).
- [22] S. Petschornig, M. Lux, and S. Chatzichristofis, "Dimensionality reduction for image features using deep learning and autoencoders," in *Proc. 15th Int. Workshop Content-Based Multimedia Indexing*, 2017, Art. no. 6, doi: [10.1145/3095713.3095737](https://doi.org/10.1145/3095713.3095737).
- [23] Nomenclature Committee of the International Union of Biochemistry (NC-IUB), "Nomenclature for incompletely specified bases in nucleic acid sequences. Recommendations 1984," *Eur. J. Biochem.*, vol. 150, pp. 1–5, 1985, doi: [10.1111/j.1432-1033.1985.tb08977.x](https://doi.org/10.1111/j.1432-1033.1985.tb08977.x).
- [24] S. Kiranyaz, A. Gastli, L. Ben-Brahim, N. Alemadi, and M. Gabbouj, "Real-time fault detection and identification for MMC using 1-D convolutional neural networks," *IEEE Trans. Ind. Electron.*, vol. 66, no. 11, pp. 8760–8771, Nov. 2019, doi: [10.1109/TIE.2018.2833045](https://doi.org/10.1109/TIE.2018.2833045).
- [25] S. Ryu, H. Choi, H. Lee, and H. Kim, "Convolutional autoencoder based feature extraction and clustering for customer load analysis," *IEEE Trans. Power Syst.*, vol. 35, no. 2, pp. 1048–1060, Mar. 2020, doi: [10.1109/TPWRS.2019.2936293](https://doi.org/10.1109/TPWRS.2019.2936293).
- [26] T. Dozat, "Incorporating Nesterov momentum into adam," 2016.
- [27] P. Gupta, R. E. Banchs, and P. Rosso, "Squeezing bottlenecks: Exploring the limits of autoencoder semantic representation capabilities," *Neurocomputing*, vol. 175, pp. 1001–1008, 2016, doi: [10.1016/j.neucom.2015.06.091](https://doi.org/10.1016/j.neucom.2015.06.091).
- [28] A. Buluç, J. T. Fineman, M. Frigo, J. R. Gilbert, and C. E. Leiserson, "Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks," in *Proc. 21st Annu. Symp. Parallelism Algorithms Architectures*, 2009, pp. 233–244, doi: [10.1145/1583991.1584053](https://doi.org/10.1145/1583991.1584053).
- [29] FPZIP: Floating-point array compressor. [Online]. Available: <https://computing.llnl.gov/projects/fpzip>
- [30] P. Lindstrom and M. Isenburt, "Fast and efficient compression of floating-point data," *IEEE Trans. Vis. Comput. Graphics*, vol. 12, no. 5, pp. 1245–1250, Sep./Oct. 2006, doi: [10.1109/TVCG.2006.143](https://doi.org/10.1109/TVCG.2006.143).
- [31] D. A. Huffman, "A method for the construction of minimum-redundancy codes," in *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952, doi: [10.1109/JRPROC.1952.273898](https://doi.org/10.1109/JRPROC.1952.273898).
- [32] B. C. L. and A. S. Nair, "Benchmark dataset for whole genome sequence compression," *IEEE/ACM Trans. Comput. Biol. Bioinf.*, vol. 14, no. 6, pp. 1228–1236, Nov./Dec. 2017, doi: [10.1109/TCBB.2016.2568186](https://doi.org/10.1109/TCBB.2016.2568186).
- [33] D. Pratas and A. J. Pinho, "A DNA sequence corpus for compression benchmark," in *Proc. 12th Int. Conf. Practical Appl. Comput. Biol. Bioinf.*, F. Fdez-Riverola, M. S. Mohamad, M. Rocha, J. F. De Paz, and P. González, Eds., Springer, 2019, pp. 208–215, doi: [10.1007/978-3-319-98702-6\\_25](https://doi.org/10.1007/978-3-319-98702-6_25).
- [34] H.-S. Kim, S. Jeon, Y. Kim, C. Kim, J. Bhak, and J. Bhak, "KOREF\_S1: Phased, parental trio-binned Korean reference genome using long reads and Hi-C sequencing methods," *Gigascience*, vol. 11, 2022, Art. no. giac022, doi: [10.1093/gigascience/giac022](https://doi.org/10.1093/gigascience/giac022).
- [35] NCBI genome datasets. [Online]. Available: <https://www.ncbi.nlm.nih.gov/data-hub/genome>



**Sheena K. S.** received the master of engineering degree in software engineering from Anna University, India, in 2009. She is currently working toward the doctoral degree in computer science and engineering with the Cochin University of Science and Technology, India. Her research interests include genomic sequence data compression, data science, and machine learning.



**Madhu S. Nair** (Senior Member, IEEE) received the master's in computer applications (MCA) (first rank) degree from Mahatma Gandhi University, in 2003, the master's in technology (MTech) (first rank) degree in computer science (with specialization in digital image computing) from the University of Kerala, in 2008, and the PhD degree in computer science (image processing) from Mahatma Gandhi University, in 2013. He also holds a post graduate diploma in client server computing (PGDCSC) with the Amrita Institute of Computer Technology. He is currently working as professor with the Artificial Intelligence & Computer Vision Lab, Department of Computer Science, Cochin University of Science and Technology (CUSAT). He has published 117 research papers in reputed International Journals and Conference Proceedings published by IEEE, Springer, Elsevier, Wiley, and IOS Press, etc. He is a senior member of the Association for Computing Machinery (ACM), associate life member of Computer Society of India (CSI), and member of the International Association of Engineers (IAENG). He is a recipient of prestigious awards such as AICTE Visvesvaraya Best Teacher Award, Kairali Gaveshana Puraskaram of Govt. of Kerala, Swami Vivekananda Yuva Prathibha Puraskaram of Govt. of Kerala, CSI Best Faculty of the Year Award, Best Thesis Supervisor Award from IEEE Communications Society, etc. His research interests include digital image processing, pattern recognition, computer vision, data compression, soft computing, and scientometrics.